

내 손으로 조작하며 배우는 5일 강화학습

260203 2일차: MDP와 파라미터 튜닝

파이썬 학습을 위해 도움이 되는 사이트

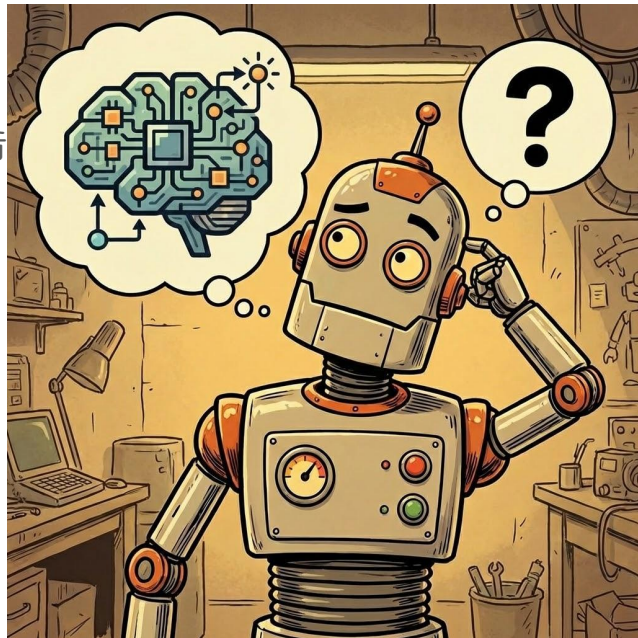
1. 점프 투 파이썬(<https://wikidocs.net/book/1>)
2. 코딩도장 (<https://dojang.io/course/view.php?id=7>)
3. 프로그래머스 실습 (<https://school.programmers.co.kr/learn/challenges>)
 - 다양한 난이도의 연습문제 제공
 - 다른 사람의 정답을 보고 학습 가능

파이썬 학습을 위해 도움이 되는 사이트

1. Google's Python Class (<https://developers.google.com/edu/python>)
2. Codecademy (<https://www.codecademy.com/catalog/language/python>)
3. Codewars (<https://www.codewars.com/>)
 - 게임화된 방식으로 코딩 문제를 풀 수 있는 사이트

오늘의 목표

- 마르코프 결정 과정(MDP)의 5가지 요소 이해
- 파라미터 최적화로 할인율(γ) 영향 체험
- MountainCarContinuous 환경을 통한 정책 실습
- Gradio로 파라미터 최적화 체험



마르코프 결정 과정 (MDP, Markov Decision Process)

- 순차적 의사결정 문제를 수학적으로 모델링하는 프레임워크
 - 환경의 '규칙'을 정의하는 방법
 - 에이전트가 '미래'를 고려하는 방법
 - 강화학습의 이론적 토대

마르코프 결정 과정의 다섯가지 요소

- S (States) - 에이전트가 있을 수 있는 모든 상태
- A (Actions) - 각 상태에서 취할 수 있는 행동
- R (Reward) - 행동에 따른 즉각적 보상
- γ (Gamma, 할인율) - 미래 보상의 현재 가치 (0~1)
- P (Transition Probability) - 상태 전이 확률 함수

할인율(γ , Gamma)이란?

- 미래 보상의 현재 가치를 결정하는 파라미터
 - $\gamma = 0$: 즉각적 보상만 고려
 - $\gamma \approx 1$: 장기적 보상을 중요하게 고려
-
- 채권의 현재 가치 결정과 비슷한 역할

누적 보상 (Cumulative Reward)

$$G = r_1 + \gamma r_2 + \gamma^2 r_3 + \gamma^3 r_4 + \dots$$

- 할인율 γ 가 미래 가치를 결정
- 에이전트는 누적 보상을 최대화하는 방향으로 학습

가치 반복법 (Value Iteration)

- 할인율(γ)을 적용해서, 목표의 보상이 전체로 퍼져나가게 계산
- 모든 타일의 가치를 0으로 시작
- 역방향 전파 (Backward Induction)
 - 다음 현재 가치 = $\text{MAX}(\text{현재 보상} + \text{할인된 인접 가치들})$
- 더 이상 값이 변하지 않을 때까지 반복

할인율(γ) 비교 실험



```
streamlit run day2/01_grid_world_overlay.py
```

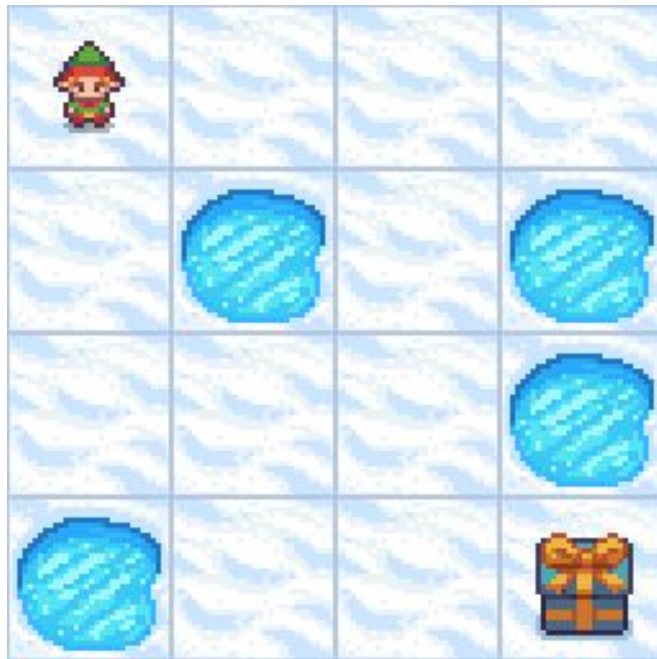
FrozenLake 환경 소개

- 얼어붙은 호수를 건너 보물을 찾는 게임

- S: 안전한 얼음 (Start)
- F: 얼어붙은 표면 (Frozen)
- H: 구멍 (Hole, 게임 종료)
- G: 목표 (Goal, 보상 +1)

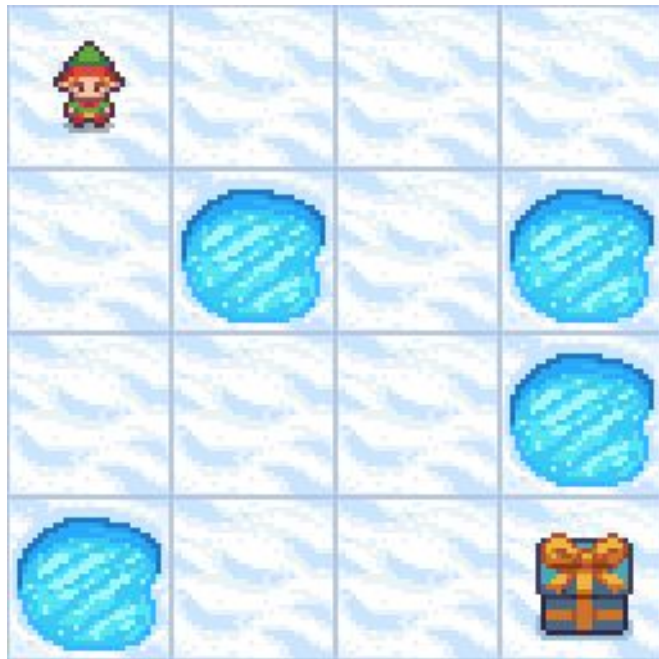
`day2/02_frozenlake_intro.ipynb`

`streamlit run day2/03_frozenlake_manual.py`



FrozenLake 환경 소개

- 얼음이 미끄러워서 의도한 것과 다르게 이동
 - 전이 확률이 불확실함 (stochastic)
- 구멍에 빠지면 게임 종료
- 목표에 도달하면 보상 획득



```
streamlit run day2/04_frozenlake_manual_slippy.py
```

전이 확률 분석

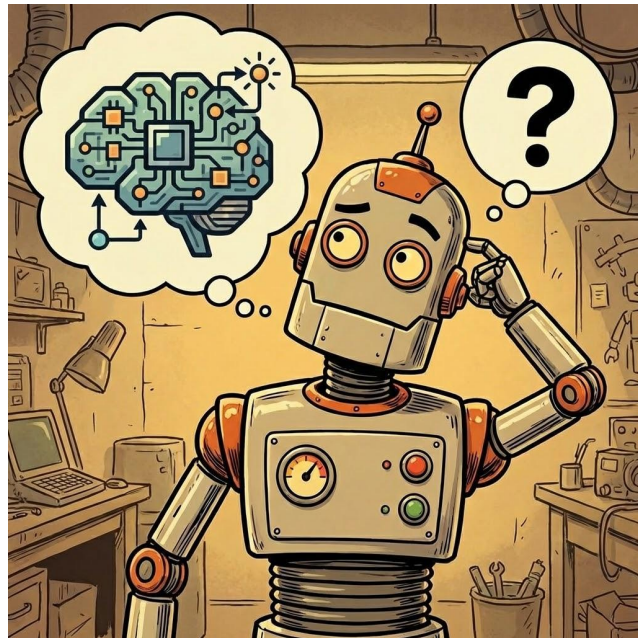
- 각 행동의 결과 예측
 - 미끄러운 얼음의 영향 이해
- 지금 상태가 **s**이고, 내가 행동 **a**를 선택했을 때,
다음 스텝의 상태가 **s'**이 될 확률은 몇 퍼센트인가?

`day2/05_env_P_analysis.ipynb`

정책 (Policy, π)

- 각 상태에서 어떤 행동을 선택할지 정하는 전략
 - 결정적 정책: $\pi(s) \rightarrow a$
 - 확률적 정책: $\pi(a|s) \rightarrow$ 확률 분포

<https://youtu.be/WXuK6gekU1Y?t=108>



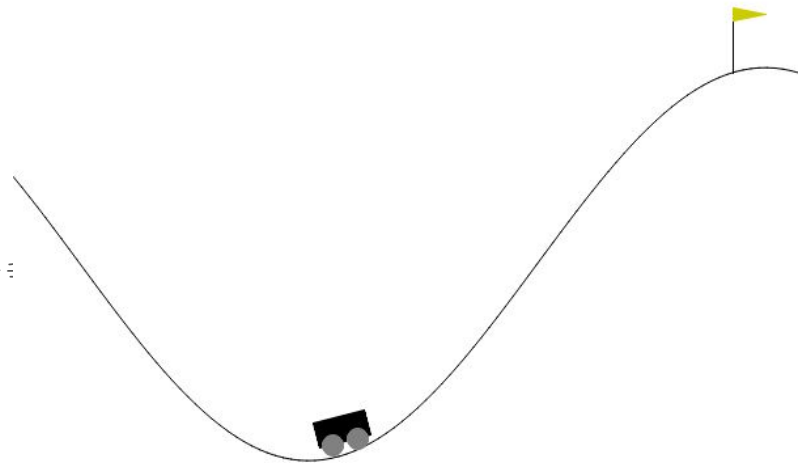
힐 클라이밍 (Hill Climbing) 정책

- 단순한 가중치 곱셈(행렬 곱) 정책
- 행동 = 상태(S)×가중치(W)+편향(b)
 - 가중치(Weight): 이 입력들에 곱해지는 단순한 숫자들
 - 편향(b): 기본 성향을 결정하는 보정값

`python day2/06_hill_climbing_cartpole.py`

Mountain Car Continuous

- 골짜기에 있는 자동차가 가속을 조절하여 언덕 정상에 있는 깃발에 도달
- 상태
 - 자동차의 현재 위치: $[-1.2, 0.6]$
 - 자동차의 현재 속도: $[-0.07, 0.07]$
- 행동
 - 가속값: $[-1.0, 1.0]$, +1.0은 오른쪽, -1.0은 왼쪽 가속
- 보상
 - 매 단계마다 일정한 페널티
 - 가속을 강하게 할수록 페널티가 커짐
 - 자동차가 깃발에 도달하면 +100의 큰 보상



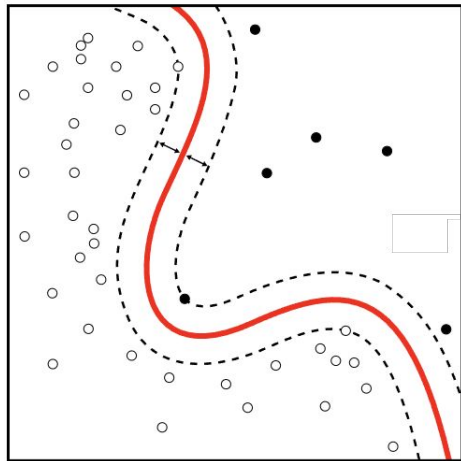
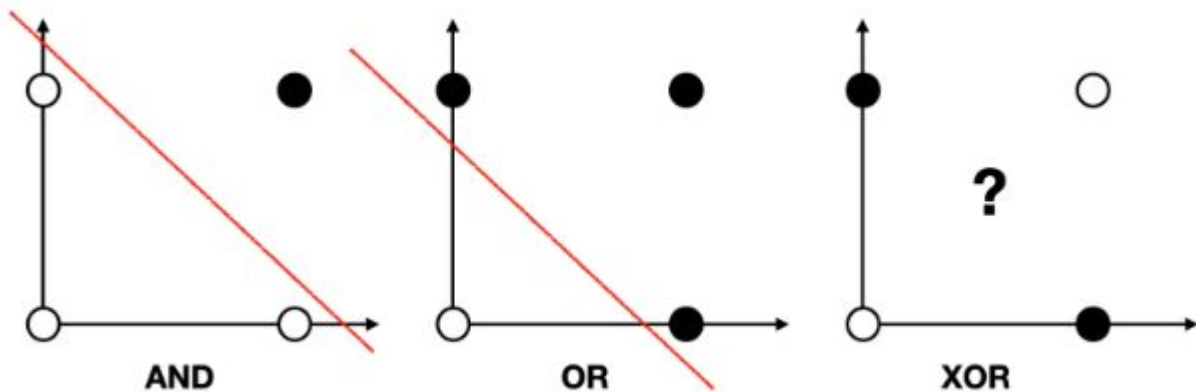
`python day2/07_hill_climbing_mountaincar.py`

힐 클라이밍 (Hill Climbing) 정책 - 단점

1. 노이즈에 민감하고 운에 따라 정책 품질이 달라진다.
2. 선형이다.
3. 지역 최적점(Local Optima)에 갇히기 쉽다.

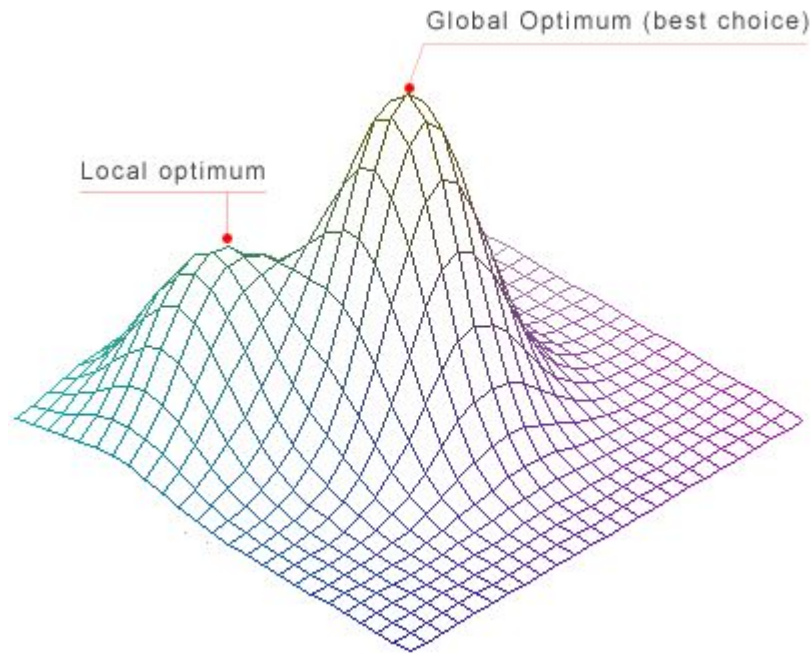
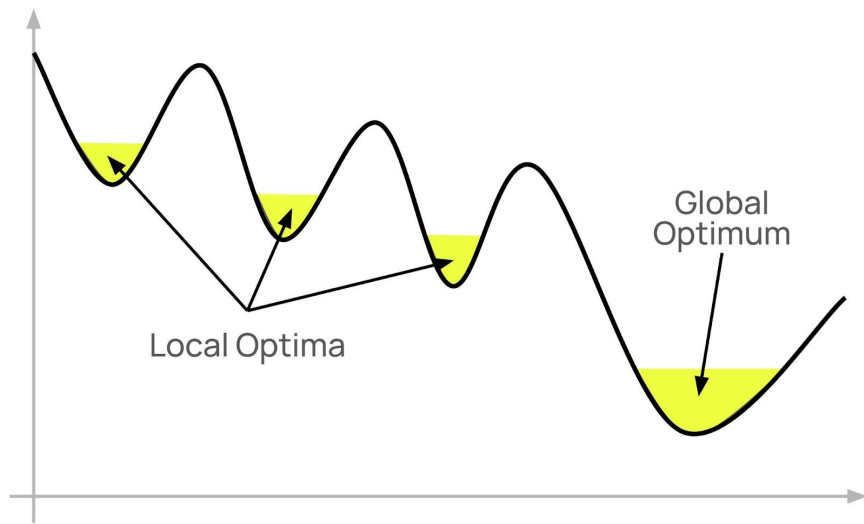
힐 클라이밍 (Hill Climbing) 정책 - 단점

- 선형이다.
 - 해들이 떨어져 있으면 불가능
 - 해들의 경계가 선형에서 멀어지면 성능이 떨어짐



힐 클라이밍 (Hill Climbing) 정책 - 단점

- 지역 최적점(Local Optima)에 갇히기 쉽다.



힐 클라이밍 (Hill Climbing) 정책 - 하이퍼 파라미터 튜닝

- 하이퍼 파라미터?
 - 모델이 스스로 학습하는 것이 아니라 사용자가 직접 설정해 주어야 하는 외부 설정값
- 모델의 구조와 학습 과정 전반을 제어
- 모델이 데이터를 얼마나 잘 학습하고 일반화할 수 있는지를 결정하는 핵심적인 요소

`day2/08_colab_gradio_cartpole.ipynb`

`day2/09_colab_gradio_mountaincar.ipynb`

Gradio 소개

- 머신러닝 모델이나 파이썬 함수를 웹 기반 인터페이스로 빠르게 구축하고 배포할 수 있도록 돕는 오픈소스 라이브러리
- 복잡한 웹 개발 지식 없이도 단 몇 줄의 파이썬 코드를 통해 직관적인 **UI**를 즉시 생성
- 텍스트, 이미지, 오디오, 비디오뿐만 아니라 **Slider, Plot, Checkbox** 등 풍부한 위젯을 제공
- 코드 변경 사항이 웹 인터페이스에 즉시 반영됨
- **TensorFlow, PyTorch, Scikit-learn** 등 주요 머신러닝 프레임워크와 완벽하게 연동

힐 클라이밍 (Hill Climbing) 정책 - 하이퍼 파라미터 튜닝

- **n_iterations**: 전체 학습 과정을 몇 번 반복할지 결정하는 총 루프 횟수
- **noise_scale**: 현재 최적의 가중치에 더할 무작위 소음(Noise)의 초기 크기로 탐색 범위를 결정
- **noise_decay**: 학습이 진행될수록 **noise_scale**을 점진적으로 줄여 정밀한 최적화를 돕는 감쇠율
- **init_scale**: 신경망의 가중치를 처음에 얼마나 큰 범위의 무작위 값으로 설정할지 결정하는 초기화 척도

오늘 배운 내용 정리

- MDP의 5가지 요소: S, A, P, R, γ
- FrozenLake 환경과 전이 확률
- 할인율의 중요성과 영향
- Streamlit, Gradio로 파라미터 최적화

3일차 예고

- Q-Table과 벨만 방정식
- epsilon-Greedy 알고리즘을 통한 데이터 수집 및 학습 효율 최적화
- 바이브 코딩으로 성능 시각화 및 검증
- 최적 정책(Optimal Policy) 학습

감사합니다